

Proyecto Git y GitHub: Biblioteca por fases

PROYECTO: SISTEMA DE BIBLIOTECA (GIT + TESTING + REFACTOR)

1. INTRODUCCIÓN

Trabajarás sobre un proyecto realista para:

- usar Git de forma profesional
- escribir tests con `unittest`
- refactorizar código
- documentar con pdoc
- trabajar en equipo (conflictos reales)

Se evalúa el proceso, no solo el resultado.

2. NORMAS OBLIGATORIAS

PROHIBIDO

- Trabajar en `main`
- Subir código sin tests previos
- Hacer un único commit final
- Reescribir todo de golpe

OBLIGATORIO

- Ramas por fase/feature
- Commits frecuentes (cada pequeño cambio)
- `git pull` antes de empezar
- Tests en TODAS las fases
- Documentación en TODAS las fases
- Trabajo simultáneo con el compañero
- Capturas de comandos Git
- Evidencias de rollback

3. FLUJO DE TRABAJO (SIEMPRE)

```
git pull
git checkout -b feature/<nombre>
# trabajar en pequeños pasos
git add .
git commit -m "mensaje claro y concreto"
git push
```

Antes de integrar:

```
git checkout main
git pull
git merge feature/<nombre>
```

4. ESTRUCTURA DEL PROYECTO

```
biblioteca/  
├── src/  
├── tests/  
├── docs/  
├── biblioteca.db  
└── README.md
```

5. FASE 1 – SETUP + ANÁLISIS

Objetivo

Preparar entorno y entender el código base.

PASOS

1. Clonar

```
git clone <URL>  
cd biblioteca
```

1. Abrir en PyCharm y ejecutar

2. Ejecutar tests

```
python-m unittest
```

3. Análisis en README (obligatorio)

- Qué hace el proyecto
- Problemas detectados
- Partes confusas
- Funciones mal diseñadas

4. Crear BD (solo esto)

```
import sqlite3  
  
conn=sqlite3.connect("biblioteca.db")  
conn.execute("PRAGMA foreign_keys = ON")  
  
conn.execute("""  
CREATE TABLE IF NOT EXISTS biblioteca (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    nombre TEXT  
)  
""")  
  
conn.commit()  
conn.close()
```

 **NO crear más tablas en esta fase.**

5. Commit

```
git checkout-b feature/setup  
git add .  
git commit-m"F1: setup, análisis y creación inicial de BD"  
git push
```

6. FASE 2 – TESTING OBLIGATORIO

Objetivo

Crear tests ANTES de modificar código.

Regla

|  No modificar código sin tests que fallen

PASOS

1. Rama

```
git checkout -b feature/testing
```

2. Crear/Completar tests en `tests/`

3. Requisitos de tests

- nombre claro
- prueban una sola cosa
- algunos deben FALLAR

1. Ejecutar

```
python -m unittest
```

2. Commit frecuente

```
git add .  
git commit -m "F2: tests para X"  
git push
```

7. FASE 3 – REFACTORIZACIÓN INICIAL

Objetivo

Mejorar código SIN añadir funcionalidad.

Regla

|  No añadir nuevas features

PASOS

1. Rama

```
git checkout -b feature/refactor
```

2. Refactor obligatorio

- renombrar variables (`x` , `data` , `tmp`)
- dividir funciones (>20 líneas)
- eliminar duplicación
- simplificar condicionales

3. Tras cada cambio

```
python-m unittest
```

4. Commits pequeños y frecuentes

```
git add .  
git commit-m"Refactor: mejora específica"
```

5. Documentación (docstrings)

- añadir docstrings en funciones modificadas
- regenerar docs con pdoc

8. FASE 4 – LIBROS (CRUD + BÚSQUEDAS)

Objetivo

Implementar entidad **Libro** y su gestión.

REQUISITOS

Clase `Libro` con:

```
id,titulo,autor,disponible,ISBN + (atributos decididos por grupo)
```

Métodos en `biblioteca.py` :

```
add_libro(libro)  
remove_libro(id)  
get_libro(id)  
list_libros()
```

BÚSQUEDAS OBLIGATORIAS (≥3)

Elegir al menos 3:

- por título
- por autor
- por disponibilidad
- por coincidencia parcial (LIKE/similar)
- por id

TESTS

- CRUD completo
- **cada tipo de búsqueda tiene su test**

DOCUMENTACIÓN

- docstrings en clase y métodos
- actualizar docs con pdoc

GIT

```
git checkout-b feature/libros  
# commits pequeños  
git push
```

9. FASE 5 – USUARIOS (CRUD + BÚSQUEDAS) [PARALELO A F4]

Objetivo

Entidad **Usuario**

Clase `Usuario` con:

```
id,nombre,apellidos,email,habilitado + (atributos decididos por grupo)
```

Métodos en `biblioteca.py`:

```
add_usuario(usuario)
remove_usuario(id)
get_usuario(id)
list_usuarios()
habilita_usuario(id)
deshabilita_usuario(id)
```

BÚSQUEDAS (≥3)

Ejemplos:

- por nombre
- por apellidos
- por email
- coincidencia parcial

TESTS

- CRUD completo
- tests por búsqueda

GIT (paralelo a libros)

```
git checkout -b feature/usuarios
```

MERGE OBLIGATORIO (LIBROS + USUARIOS)

```
git checkout main
git pull
git merge feature/libros
git merge feature/usuarios
```

Resolver conflictos en `biblioteca.py`

10. FASE 6 – PRÉSTAMOS (TRABAJO EN PARALELO)

Objetivo

Relacionar libros y usuarios

FUNCIONES

```
prestar_libro(libro_id,usuario_id)
devolver_libro(libro_id,usuario_id)
```

⚠ TRABAJO EN PAREJA

- Alumno A → `prestar_libro`
- Alumno B → `devolver_libro`

👉 en paralelo, misma zona → conflicto seguro

🔧 TESTS

- prestar correctamente
- no prestar si no existe
- no prestar si no disponible
- devolver correctamente

📄 DOCUMENTACIÓN

- docstrings + pdoc

11. FASE 7 – LOGS

🎯 Objetivo

Registrar acciones

Ejemplo:

```
"Usuario X ha prestado Libro Y"
```

🔧 TESTS

- comprobar registro de acciones

12. FASE 8 – INTEGRACIÓN TOTAL

🎯 Objetivo

Unificar en:

```
operar(accion,datos)
```

💥 Conflictos garantizados

13. ROLLBACK OBLIGATORIO

Debes demostrar:

🔄 Caso 1 – revert

```
git revert <commit>
```

🔄 Caso 2 – reset

```
git reset--hard <commit>
```

📷 OBLIGATORIO

- capturas de:

- commits
- conflictos
- rollback

14. ENTREGABLES

PDF de documentación

Dicho PDF seguirá todos los pasos realizados en el proyecto con detalles y explicaciones además de muchas y variadas capturas de pantalla.

Este PDF podrá ser usado como herramienta de ayuda en la defensa del proyecto.

Código

- funcional
- testeado
- documentado

Evidencias (OBLIGATORIO)

- capturas de:
 - `git log`
 - conflictos
 - merges
 - rollbacks

README debe incluir

- Cómo ejecutar
- Cómo testear
- Qué problemas hubo
- Cómo resolviste conflictos
- Qué refactorizaciones hiciste

15. RECOMENDACIONES (OBLIGATORIAS EN PRÁCTICA)

- Haz commits pequeños constantemente
- Haz `pull` antes de trabajar
- Ejecuta tests tras cada cambio
- No acumules cambios grandes
- Documenta mientras desarrollas
- Coordínate con tu compañero

16. RÚBRICA GLOBAL – PROYECTO BIBLIOTECA

Leyenda de niveles

Código	Nivel	Descripción general
0	 No conseguido	No hay evidencia o es incorrecto

Código	Nivel	Descripción general
1	Deficiente	Uso mínimo, incorrecto o superficial
2	Insuficiente	Uso básico con errores importantes
3	Aceptable	Uso correcto pero limitado
4	Adecuado	Buen uso, consistente
5	Correcto	Uso excelente, completo y profesional

Criterio	Peso	0 	1 	2 	3 	4 	5 
1. Git 	15%	Sin uso o incorrecto	Uso mínimo: commits aislados, sin ramas ni flujo	Uso básico: ramas poco claras, pocos commits, sin variedad de comandos	Uso correcto: commits frecuentes, ramas, <code>pull</code> , <code>merge</code> básicos	Buen uso: commits granulares, ramas organizadas, uso de <code>merge</code> , <code>pull</code> , <code>log</code> , <code>.gitignore</code> , conflictos resueltos	Uso profesional: commit granular, ramas con uso variado de comandos (<code>merge</code> , <code>rebase</code> , opciones <code>pull</code> , <code>.gitignore</code> , <code>stash</code> , opciones <code>rollback</code> (<code>revert</code> , <code>reset</code>), <code>correct</code> aplicadas, evidencia múltiple, conflicto resuelto correctamente.
2. Testing 	12.5%	Sin tests	Tests irrelevantes o inexistentes en clases clave	Tests básicos en funciones sueltas, baja cobertura	Tests correctos en clases gestoras, cobertura media	Tests en clases gestoras y clases de objetos , buena cobertura, casos de error incluidos	Cobertura amplia: todas las capas (+ gestión de calidad), casos con buena validación, errores, independencia de tests, justificación clara.
3. Documentación 	12.5%	Sin documentación	Muy pobre	Parcial	Correcta	Completa	Excelencia clara, coherencia profesional.
4. PDF	15%	No entregado	Muy pobre	Básico	Correcto	Completo	Excelencia detallada, evidencia (commit, conflicto, rollback).
5. Defensa 	20%	No presenta	Muy pobre	Inseguro	Correcto	Buen dominio	Dominio argumentado sólido.
6. Funcionalidades	10%	No hay	Trivial	Simple	Útil	Interesante	Completo, bien integrado.

Criterio	Peso	0 	1 	2 	3 	4 	5 
nuevas							
7. Código	10%	Caótico	Confuso	Mejorable	Claro	Bien estructurado	Limpio, escalab profesic
8. Colaboración	-15% / +5%	-15%	-10%	-5%	0%	+3%	+5%
9. Calidad global	+10%	0%	+2%	+4%	+6%	+8%	+10%